

Watching the Internet Knock

A 25-hour SSH & RDP honeypot on a bare public cloud VM — what automated attackers actually try, how fast they find you, and what it reveals about the background noise every exposed server on the internet lives in.

9,808

Connection attempts logged in
~25 hours

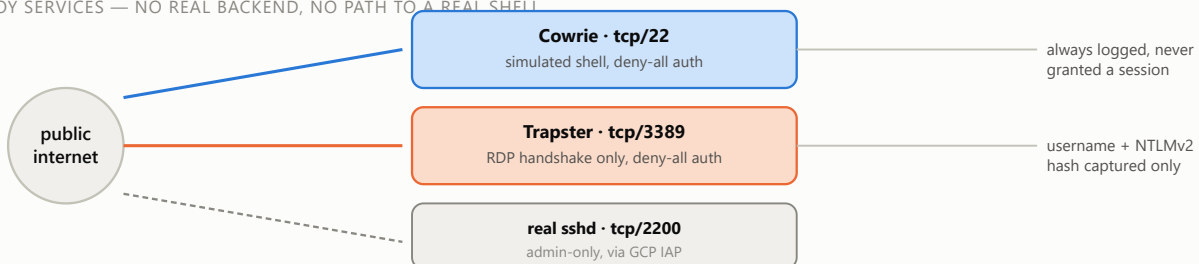
17 sec

Time from deployment to first RDP probe

201

Unique source IPs, 60+ countries

DECOY SERVICES — NO REAL BACKEND, NO PATH TO A REAL SHELL



**Tomer
Shapira**
Author

August 2026
Study window: Aug 16–17,
2026 (UTC)

**GCP e2-small ·
Debian 12**
Cowrie (SSH) +
Trapster (RDP)

github.com/tomershapira103/Honeypot
Code & full methodology

Why point a decoy at the open internet?

Any server with a public IP starts receiving unsolicited connection attempts almost immediately — long before a human ever finds it. That traffic is usually invisible: it lives in auth logs nobody reads, on production boxes where "someone tried to log in and failed" isn't interesting enough to investigate. This project made that traffic the entire point.

I stood up a small cloud VM running two protocol-level decoys — nothing behind them, no real filesystem, no real remote desktop, no path to an actual shell — and let the internet find it on its own. Every connection, every credential guess, and every client fingerprint was logged and analyzed to answer four concrete questions:

1 How long until first contact?

From the moment the VM goes live, how many seconds or hours pass before the first scanner finds each open port?

2 How much traffic, sustained over time?

Is this a one-time sweep or a continuous, ongoing background hum of scanning?

3 What credentials do attackers actually try?

Which usernames and passwords dominate real-world brute-force dictionaries in 2026?

4 What's doing the knocking?

Generic botnet tooling, purpose-built scanners, or something closer to a human with a terminal?

WHY THIS MATTERS

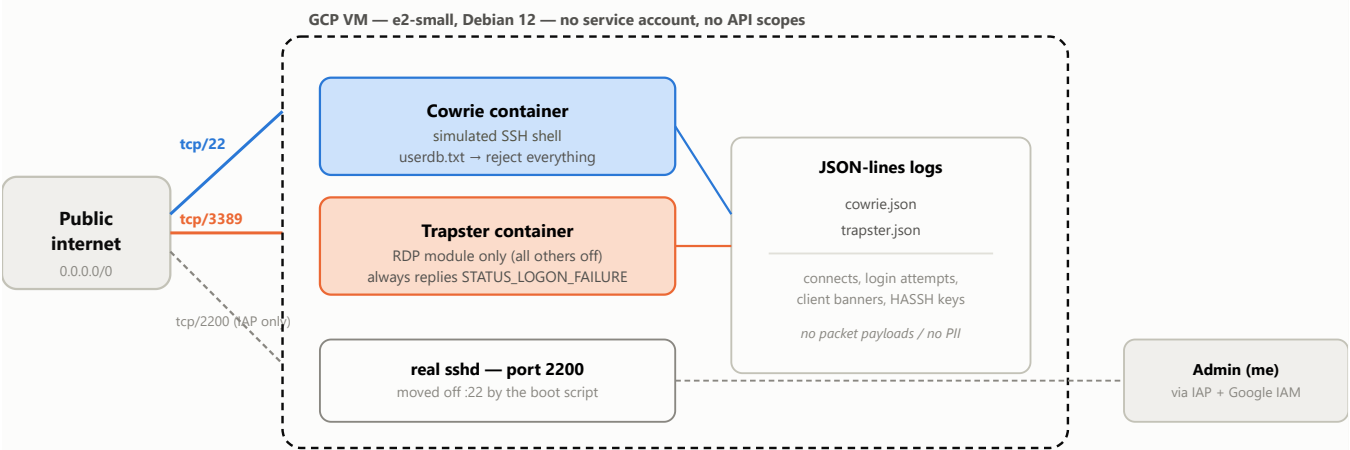
Every one of these connection attempts is aimed at the same numbers ports on *every* internet-facing box, all the time. Seeing the shape of that traffic — its speed, its volume, its credential lists — is directly useful for anyone hardening a server, sizing an intrusion-detection baseline, or just deciding whether "nobody would bother attacking my tiny VM" is a safe assumption. (It isn't.)

Ground rules for the study

- **Protocol-level decoys only.** Cowrie presents a simulated Linux shell over SSH; Trapster completes just enough of the RDP handshake to capture a username. Neither has a real backend — an attacker can never get further than a login prompt.
- **Deny-all authentication.** Every credential is rejected, on both services, always. This maximizes the number of distinct guesses captured, at the cost of never observing post-login behavior (see Limitations, p.10).
- **Admin access preserved separately.** Real SSH access is moved to a non-standard port, reachable only through an authenticated cloud tunnel — never exposed to the ports under study.

How the decoy was built

The setup runs on a single small VM in Google Cloud, provisioned by an idempotent shell script that creates an isolated VPC, opens exactly the ports under study, and deploys two containerized honeypots.



Every resource-creation step in the provisioning script is idempotent, so a partial run can simply be re-run. Analysis is a separate, offline step run against the downloaded logs.

Design choices worth calling out

- **Real admin access never touches the studied ports.** SSH for administration lives on port 2200, reachable only through Google's Identity-Aware Proxy — authenticated by IAM identity, not source IP, so it keeps working from anywhere with nothing to allowlist.
- **Blast radius is capped at the VM.** The instance runs with `--no-service-account --no-scopes`, so even a hypothetical full compromise of the box has no credentials to reach any other cloud resource.
- **RDP passwords are never actually plaintext.** RDP's NLA/CredSSP handshake encrypts the password before it's sent. Trapster captures a plaintext *username* pre-TLS, plus (if the client completes NLA) a **NetNTLMv2 hash** — offline-crackable, but not a readable password. The analysis keeps these labeled and separate from SSH's genuinely plaintext credentials.

Deployed	2026-08-16 19:34:45 UTC
Observation window	~25 hours
SSH honeypot	Cowrie (medium-interaction)
RDP honeypot	Trapster Community (RDP module)
VM	GCP e2-small, Debian 12
Analysis	analysis/analyze_logs.py (Python)

The internet found it before I'd finished deploying

The most immediate finding wasn't a credential or a client fingerprint — it was the clock. Mass scanning of the public IPv4 space is fast enough that "newly provisioned" and "already being probed" are nearly the same moment.

FIRST SSH (:22) PROBE

4h 27m

after the VM went live

FIRST RDP (:3389) PROBE

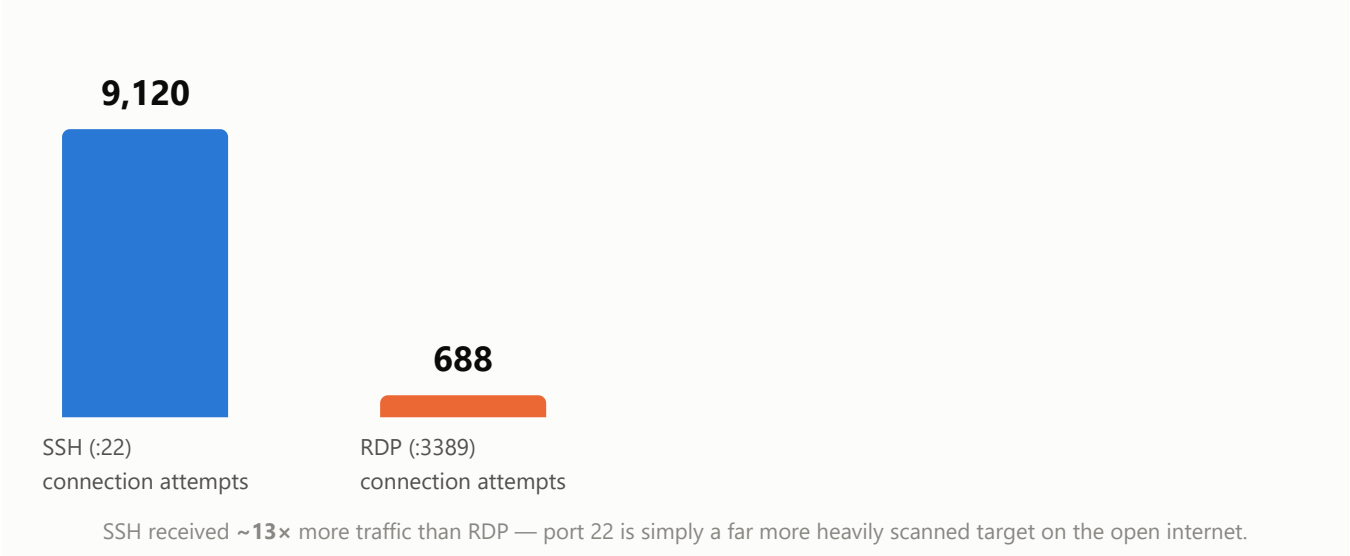
17 seconds

after the VM went live

READING THE GAP

RDP was found essentially instantly — consistent with continuous, IPv4-space-wide scanning services (some of the earliest RDP connections in the logs trace back to known internet-scanning operators). SSH took over four hours, likely because port 22 is scanned by a much larger and more varied population of tools with different sweep schedules — the first hit is closer to "whichever scanner's rotation reaches this /24 next" than a fixed latency.

Total volume over the ~25-hour window



Day-by-day breakdown

PORT	FIRST 24H	REMAINING ~53 MIN OF THE WINDOW	TOTAL	UNIQUE SOURCE IPS
22/ssh	9,075	45	9,120	143
3389/rdp	612	76	688	60

The RDP uptick in the final, partial window traces mostly to a single aggressive source performing a real credential-guessing run (see p.6) — a reminder that short observation windows can be skewed by one active actor.

Traffic is concentrated, not evenly spread

On SSH, the busiest **10 of 143** unique source IPs (7% of sources) account for **66%** of all traffic. Averaged across all sources, each SSH IP made ~64 attempts — this isn't 143 people trying once each, it's a smaller number of persistent, repeat-hitting scanners.

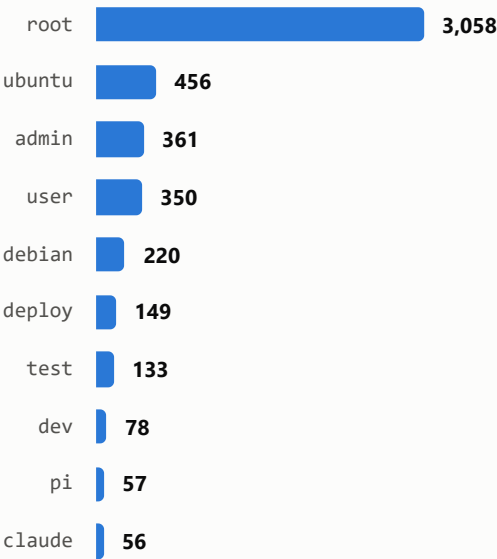
On RDP, it's even more concentrated: a **single source IP** (UK-hosted) accounts for **44.5%** of all RDP traffic on its own, and the top 3 IPs cover **70%** — RDP scanning in this window was dominated by a handful of determined actors rather than broad, diffuse noise.

What attackers type when they think no one's watching

Across the SSH honeypot, attackers tried **3,901 distinct username/password combinations** over **8,928 login attempts** — an average of roughly 2.3 tries per unique pair, consistent with automated dictionary attacks rather than manual guessing.

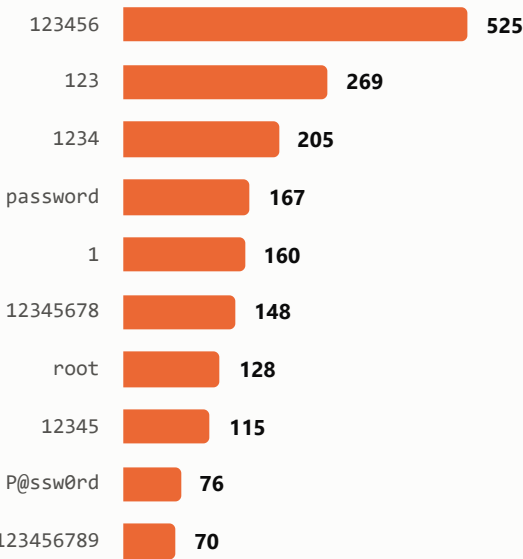
Top 10 usernames tried (SSH)

by total login attempts, all passwords combined



Top 10 passwords tried (SSH)

by total login attempts, all usernames combined



1 in 3 SSH login attempts (34.3%) used the username `root` — by far the single most-targeted account on any exposed Linux box.

UNEXPECTED FIND — THE DICTIONARIES HAVE KEPT UP WITH 2026

Buried in the username list: `claudio` (56 attempts), `openclaw` (38), and `codex` (12) — all names associated with AI coding agents / AI-assisted dev tooling. Generic brute-force wordlists are evidently being refreshed to guess at accounts an "AI dev box" might plausibly have, right alongside classics like `ubuntu` and `admin`.

RDP: usernames without passwords

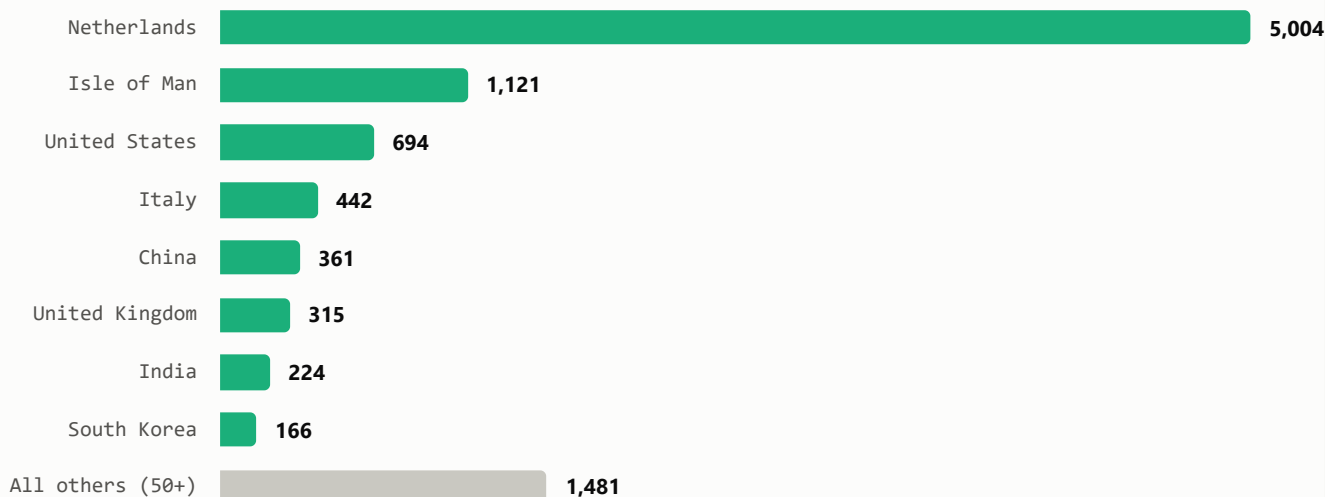
RDP's NLA/CredSSP handshake never sends a readable password. Trapster captured **330 unique usernames** across **878 login-stage events** — but the large majority are single pre-TLS username probes with no follow-through (top guesses: `test`, `workgroup`, `win-rdp01`, `hello`). Only **one source IP** completed full NLA negotiation and ran an actual brute-force, submitting **administrator** against 13 distinct guessed passwords as recoverable NetNTLMv2 hashes — everything else in the RDP data is reconnaissance, not a real login attempt.

A global footprint, but a concentrated core

Source IPs geolocated to **60+ countries** — but the raw country ranking is dominated by a small number of hosting providers running many scanner instances from adjacent IP blocks, not by broad, organic geographic spread.

Connection attempts by country of origin

combined SSH + RDP, top 8 countries + all others (aggregated by IP geolocation)



THE NETHERLANDS NUMBER IS THREE HOSTING ACCOUNTS, NOT A COUNTRY TREND

93% of the Netherlands total (4,668 of 5,004 attempts) comes from just **two ISPs** running clusters of IPs in the same block: six co-located addresses from one provider and three from another, each individually making 400–780 attempts. Geolocation tells you where the server rack is, not who's operating it — treat country-level rankings as an infrastructure map, not an attacker demographic.

A quieter signal: legitimate internet-wide scanners

Some of the traffic isn't attack traffic at all. IP ranges belonging to **Censys** and known internet-mapping infrastructure appear in both the SSH and RDP logs, and one SSH connection carried the client string `MGLNDD_<ip>_22` — a signature associated with commercial internet-scanning platforms. This is a useful reminder when reading any honeypot's totals: a meaningful slice of "attacker" traffic is research/attack-surface-mapping scanners cataloguing the internet, indistinguishable at the network layer from hostile recon.

A note on what's *not* published here

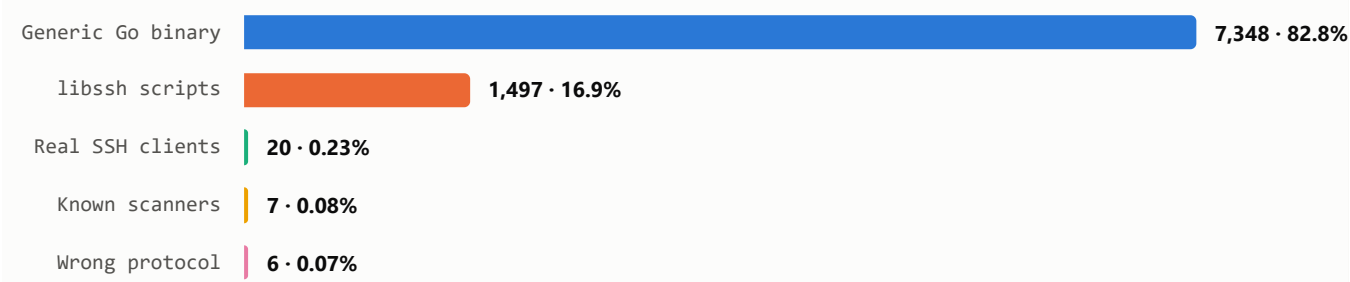
Individual source IP addresses are deliberately omitted from this report (consistent with how the underlying project handles this data — raw attacker IPs and credentials are excluded from version control by design). Everything above is aggregated to the country/ISP level, which preserves the pattern without publishing a list of specific hosts.

Almost none of this is a human at a keyboard

Cowrie logs each SSH client's raw identification banner and a HASSH fingerprint derived from its key-exchange proposal — together, a good proxy for "what software is on the other end of this connection."

SSH client tooling, by category (8,878 fingerprinted connections)

grouped from raw client version banners captured by Cowrie



Generic Go binary (82.8%) — connections announcing themselves simply as `SSH-2.0-Go`, i.e. built with Go's standard SSH library and nothing else identifying. This is the signature of custom-compiled scanning/credential-stuffing tools, likely botnet agents — by far the dominant source of traffic. Four distinct HASSH key-exchange fingerprints appear within this banner alone, meaning it's several different tools or campaigns, not one.

libssh scripts (16.9%) — almost entirely one version (`libssh_0.9.6`), the library behind countless small brute-force scripts distributed online. Its near-total dominance within this slice suggests one popular script or tutorial being run at scale.

Real interactive clients (0.23%) — `paramiko`, `PuTTY`, `russh`, one legacy `OpenSSH` — the kind of tooling an actual person or a legitimate automation script would use. Vanishingly rare against this backdrop.

Known scanners (0.08%) — recognizable research/recon tools: `ZGrab`, `Nmap`'s SSH algorithm enumerator, `Fingerprintx`.

Wrong protocol (0.07%) — a handful of connections sent a raw HTTP `GET /` request, or raw TLS `ClientHello` bytes, at port 22 — scanners blasting every protocol at every port without bothering to check what's listening.

THE HEADLINE

Over **99.9%** of fingerprinted SSH traffic came from generic, non-interactive tooling. In 25 hours, essentially zero traffic looked like "a person opened a terminal and typed `ssh`." Everything hitting this box was automated — a fair proxy for what any newly provisioned, internet-facing SSH server should expect from minute one.

What this changes about how I think about exposure

1

"Nobody will find my small server" is false, and fast.

RDP was probed 17 seconds after the VM came online. Any public IP is discovered by continuous internet-wide scanning within minutes to hours — obscurity through a low profile buys you nothing on the timescale of scanning.

2

The volume gap between SSH and RDP is real and large.

Port 22 saw ~13× the traffic of port 3389 in the same window. If you can only harden or closely monitor one exposed service, SSH is the busier target.

3

Default and weak credentials remain the entire attack.

`root / 123456` -class guesses dominate; there's no sophistication here to defend against — disabling password auth in favor of keys neutralizes nearly all of this traffic in one config change.

4

Brute-force wordlists are quietly evolving with the industry.

Seeing `claude`, `codex`, and `openclaw` as guessed usernames suggests dictionaries are being refreshed for accounts an "AI-assisted dev box" plausibly has — worth remembering if you name service accounts after the tools running on them.

5

Traffic is concentrated, not diffuse.

A handful of source IPs and hosting providers account for the majority of attempts on both ports. Rate-limiting or blocking a small number of persistent, repeat offenders would meaningfully cut the noise a real server sees.

6

This is almost entirely machine-on-machine.

Over 99.9% of fingerprinted SSH clients were generic automated tooling. The mental model of "an attacker" here is closer to "one of a few dozen running bots" than "a person deciding to target you."

Limitations & where to find everything

Limitations

- A single ~25-hour window on one cloud provider, one region, and one IP — traffic volume and mix will vary with time, provider, and IP reputation/history.
- Deny-all authentication was a deliberate choice to maximize distinct credentials captured — it means **no post-login behavior** (commands run, payloads dropped) was observed. That's a natural next study, run as a separate, clearly-scoped instance.
- Country/ISP attribution is IP geolocation, which reflects hosting infrastructure, not necessarily the operator's actual location.
- RDP's NLA/CredSSP design means most "password" data captured is either absent (pre-auth username only) or an offline-crackable hash, not a verified plaintext credential.

Code & full methodology

The full source — provisioning scripts, honeypot configuration, and the analysis pipeline used to produce every number in this report — is public:

github.com/tomershapira103/Honeypot